Day 13 Task Workflow — Tuesday, Mar 25, 2026

> **Sprint:** 3 (Intelligence Integration) | **Phase:** IOC Correlation + LLM Auto-Narrative + WebSocket Broadcasting
> **Owner:** Lead Architect | **Status:** ⬤ Ready to Start
> **Goal:** LLM auto-narrative on alerts, IOC Correlation Engine, POST /ml/retrain, WebSocket broadcasting, hyperparameter tuning
> **Grade:** Week 3 Day 3 COMPLETE ☑ | Week 3 Day 4 STARTING ⬤

Day 12 Results Context (Critical)

E2E pipeline is LIVE on VPS. Day 12 bug fixes have been deployed:

```
Container Status (Day 12 Final):
  tm-backend     ☑ Rebuilt with Day 12 fixes
  tm-capture     ☑ Up 2+ days (63 features per flow)
  tm-ml-worker   ☑ Up 21+ hours — 22,700+ flows scored
  tm-postgres    ☑ Healthy 3 days
  tm-redis       ☑ Healthy 3 days

Day 12 Bugs Fixed:
  ☑ Invalid OpenRouter model IDs → nvidia/nemotron-3-super-120b-a12b:free
restored as primary
  ☑ Alert duplicate key constraint → UUID+timestamp format (TM-YYYYMMDDHHMMSS-
XXXXXXXX)
  ☑ Missing DB columns → 4 Float columns added to Alert model (composite_score,
if_score, rf_score, ae_score)
  ☑ Empty LLM content → Returns meaningful error message

LLM Gateway (3 verified models):
  ☑ nvidia/nemotron-3-super-120b-a12b:free  → Complex analysis (primary)
  ☑ openai/gpt-oss-120b:free                → Chat / General
  ☑ stepfun/step-3.5-flash:free             → Real-time / Translation

Live Detection (Real-world validation):
  16:30:27 [Worker] ALERT: MEDIUM — probe (score=0.52, agreement=majority)
  16:58:05 [Worker] ALERT: MEDIUM — probe (score=0.52, agreement=majority)
```

⚠ PREREQUISITE: Database Migration (Must Run FIRST)

Before any Day 13 tasks, the following migration MUST be executed on VPS:

```sql
-- Run on VPS PostgreSQL:
ALTER TABLE alerts ALTER COLUMN alert_id TYPE VARCHAR(50);
ALTER TABLE alerts ADD COLUMN IF NOT EXISTS composite_score FLOAT;
```

```
ALTER TABLE alerts ADD COLUMN IF NOT EXISTS if_score FLOAT;
ALTER TABLE alerts ADD COLUMN IF NOT EXISTS rf_score FLOAT;
ALTER TABLE alerts ADD COLUMN IF NOT EXISTS ae_score FLOAT;
```

Then rebuild containers to pick up alert_engine code changes:

```
docker compose build --no-cache backend ml-worker
docker compose up -d
```

Verify:

```
SELECT alert_id, severity, category, composite_score, if_score, rf_score,
ae_score, created_at
FROM alerts WHERE alert_id LIKE 'TM-202%' ORDER BY created_at DESC LIMIT 5;
```

Scope Adherence Checklist

Requirement	Source Document	Section	Deviation?
LLM auto-narrative on alert creation	MASTER_DOC_PART4	§12.1 step [6]→[7]	☑ No change
IOC Correlation Engine	MASTER_DOC_PART4	§11.3	☑ No change
POST /ml/retrain endpoint	MASTER_DOC_PART2	§5.1	☑ No change
WebSocket new_alert broadcast	MASTER_DOC_PART2	§5.2 (alerts:live → new_alert)	☑ No change
WebSocket anomaly_detected broadcast	MASTER_DOC_PART2	§5.2 (ml:live → anomaly_detected)	☑ No change
Ensemble weights (0.30/0.45/0.25)	MASTER_DOC_PART4	§1.2	🔒 LOCKED
Alert thresholds (0.90/0.75/0.50/0.30)	MASTER_DOC_PART4	§1.2	🔒 LOCKED
LLM via OpenRouter only	SESSION_HANDOFF	Confirmed deviation	⚠ Confirmed
Prompts follow PART4 §9.2 templates	MASTER_DOC_PART4	§9.2	☑ No change

---

Day 13 Objective

By end of day:

- AlertEngine triggers async LLM narrative on every persisted alert → UPDATE alert.ai_narrative
- IOC Correlation Engine matches live flow IPs against threat_intel_iocs table
- POST /ml/retrain endpoint implemented (triggers model retraining via background task)
- WebSocket broadcasts new_alert + anomaly_detected events to connected clients
- Hyperparameter tuning executed on VPS (tune_models.py)
- E2E test traffic generated with nmap/hping3 to validate full pipeline

> **NOTE:** All manual testing and VPS verification performed by Lead Architect.

LOCKED CONSTRAINTS (DO NOT MODIFY)

```
Ensemble Weights:
  composite = 0.30 × IF + 0.45 × RF + 0.25 × AE

Alert Thresholds:
  ≥ 0.90 → CRITICAL
  ≥ 0.75 → HIGH
  ≥ 0.50 → MEDIUM
  ≥ 0.30 → LOW
  < 0.30 → NONE

LLM Provider: OpenRouter only (3 verified models)
DO NOT suggest: Kafka, Kubernetes, Elasticsearch, MongoDB
```

Task Breakdown

TASK 1 — LLM Auto-Narrative on Alert Creation 🔴

Time Est: 90 min | **Priority:** 🔴 Critical
Source: MASTER_DOC_PART4 §12.1 steps [6]→[7]

When AlertEngine persists a new alert to PostgreSQL, it should asynchronously call LLMGateway.analyze_alert() and UPDATE the alert's `ai_narrative` column with the generated text.

1.1 Modify AlertEngine._process_alert()

After the INSERT succeeds, fire-and-forget an async task:

```
# In alert_engine.py, after session.commit():

# Fire async LLM narrative generation (non-blocking)
asyncio.create_task(
    self._generate_narrative(alert_ref, payload)
)
```

1.2 Add _generate_narrative() Method

```
async def _generate_narrative(self, alert_id: str, payload: Dict[str, Any]) ->
None:
    """
    Async LLM narrative generation for a new alert.
    Calls LLMGateway.analyze_alert() → UPDATE alert.ai_narrative.
    Non-blocking: failure does not block alert persistence.
    """
    try:
        from app.services.llm_gateway import LLMGateway, TaskType

        # Build alert data for the prompt template (per PART4 §9.2)
        alert_data = {
            "severity": payload.get("severity", "medium"),
            "category": payload.get("category", "anomaly"),
            "source_ip": payload.get("source_ip", "unknown"),
            "dest_ip": payload.get("dest_ip", "unknown"),
            "confidence": payload.get("composite_score", 0.0),
            "model_agreement": payload.get("model_agreement", "unknown"),
            "if_score": payload.get("if_score", 0.0),
            "rf_label": payload.get("rf_label", "unknown"),
            "rf_confidence": payload.get("rf_confidence", 0.0),
            "ae_score": payload.get("ae_score", 0.0),
            "composite_score": payload.get("composite_score", 0.0),
        }

        gateway = LLMGateway()
        result = await gateway.analyze_alert(alert_data)
        narrative = result.get("content", "")

        if narrative and not narrative.startswith("["):
            # UPDATE alert.ai_narrative in PostgreSQL
            async with async_session() as session:
                update_sql = text("""
                    UPDATE alerts
                    SET ai_narrative = :narrative, updated_at = :updated_at
                    WHERE alert_id = :alert_id
                """)
                await session.execute(update_sql, {
                    "narrative": narrative,
                    "alert_id": alert_id,
                    "updated_at": datetime.now(timezone.utc),
                })
                await session.commit()

            logger.info("[AlertEngine] LLM narrative saved for %s (%d chars)",
alert_id, len(narrative))
        else:
            logger.warning("[AlertEngine] LLM returned empty/error for %s",
alert_id)
```

```
        await gateway.close()

    except Exception as e:
        logger.error("[AlertEngine] Narrative generation failed for %s: %s",
alert_id, e)
```

1.3 Design Decisions

- **Fire-and-forget:** Alert persistence MUST NOT be blocked by LLM latency/failure.
- **Singleton concern:** The LLMGateway import is inside the method to avoid circular imports from the ML Worker context. In production, a shared singleton should be injected.
- **Idempotency:** If narrative fails, the alert still exists with `ai_narrative = NULL`. Can be retried.
- **Rate limiting:** OpenRouter free tier has rate limits. If many alerts fire simultaneously, some narratives may fail — acceptable for dev.

Verification:

#	Check	Expected
1	Alert persists without waiting for LLM	INSERT completes in <50ms
2	LLM narrative appears in alert row	ai_narrative column NOT NULL after ~5-15s
3	LLM failure doesn't crash AlertEngine	Error logged, alert still saved
4	Narrative uses correct prompt template	References ML scores, severity, category

---

TASK 2 — IOC Correlation Engine 🟢

Time Est: 75 min | **Priority:** 🟢 Medium
Source: MASTER_DOC_PART4 §11.3

Match live flow source/destination IPs against the `threat_intel_iocs` table. Auto-escalate alert severity if a flow involves a known malicious IP.

2.1 Create IOC Correlator

File: `backend/app/services/ioc_correlator.py`

```
"""
ThreatMatrix AI — IOC Correlation Engine

Per MASTER_DOC_PART4 §11.3:
  1. Check src_ip / dst_ip against threat_intel_iocs table
  2. If match → auto-escalate alert severity
  3. If dst_domain (DNS) matches → flag as C2/phishing
"""

from __future__ import annotations
```

```python
import logging
from typing import Any, Dict, List, Optional

from sqlalchemy import text
from sqlalchemy.ext.asyncio import AsyncSession

from app.database import async_session

logger = logging.getLogger(__name__)


class IOCCorrelator:
    """
    Correlates live network flow IPs against stored IOCs.
    """

    async def check_ip(self, ip_address: str) -> Optional[Dict[str, Any]]:
        """
        Check if an IP exists in the threat_intel_iocs table.
        Returns IOC data if found, None otherwise.
        """
        async with async_session() as session:
            result = await session.execute(
                text("""
                    SELECT ioc_value, threat_type, severity, source,
                           confidence, tags, first_seen, last_seen
                    FROM threat_intel_iocs
                    WHERE ioc_type = 'ip'
                      AND ioc_value = :ip
                      AND is_active = true
                    ORDER BY confidence DESC
                    LIMIT 1
                """),
                {"ip": ip_address},
            )
            row = result.fetchone()
            if row:
                return {
                    "ioc_value": row[0],
                    "threat_type": row[1],
                    "severity": row[2],
                    "source": row[3],
                    "confidence": row[4],
                    "tags": row[5],
                    "first_seen": str(row[6]) if row[6] else None,
                    "last_seen": str(row[7]) if row[7] else None,
                }
        return None

    async def correlate_flow(self, flow_data: Dict[str, Any]) -> Dict[str, Any]:
        """
        Correlate a flow's source and destination IPs against IOC database.

        Returns:
```

```
            {
                "src_match": {...} or None,
                "dst_match": {...} or None,
                "has_ioc_match": bool,
                "escalation_severity": str or None,
            }
        """
        src_ip = flow_data.get("source_ip") or flow_data.get("src_ip")
        dst_ip = flow_data.get("dest_ip") or flow_data.get("dst_ip")

        src_match = await self.check_ip(src_ip) if src_ip else None
        dst_match = await self.check_ip(dst_ip) if dst_ip else None

        has_match = src_match is not None or dst_match is not None

        # Determine escalation per PART4 §11.3
        escalation = None
        if has_match:
            # Use the higher severity from either match
            match_severity = (src_match or dst_match or {}).get("severity",
"medium")
            if match_severity == "critical":
                escalation = "critical"
            elif match_severity == "high":
                escalation = "high"
            else:
                escalation = "high"  # Any IOC match → at least HIGH

        return {
            "src_match": src_match,
            "dst_match": dst_match,
            "has_ioc_match": has_match,
            "escalation_severity": escalation,
        }

    async def bulk_check(self, ip_list: List[str]) -> Dict[str, Optional[Dict]]:
        """Check multiple IPs at once."""
        results = {}
        for ip in ip_list:
            results[ip] = await self.check_ip(ip)
        return results
```

2.2 Integrate IOC Correlation into AlertEngine

After alert INSERT, before LLM narrative:

```
# In _process_alert(), after INSERT:
from app.services.ioc_correlator import IOCCorrelator

correlator = IOCCorrelator()
ioc_result = await correlator.correlate_flow(payload)
```

```
if ioc_result["has_ioc_match"]:
    # Auto-escalate severity
    escalated_severity = ioc_result["escalation_severity"]
    async with async_session() as session:
        await session.execute(
            text("UPDATE alerts SET severity = :sev WHERE alert_id = :aid"),
            {"sev": escalated_severity, "aid": alert_ref},
        )
        await session.commit()
    logger.info("[AlertEngine] IOC match! Escalated %s → %s", alert_ref,
escalated_severity)
```

Verification:

#	Check	Expected
1	IOCCorrelator.check_ip("8.8.8.8")	None (clean IP)
2	Insert test IOC, then check	Returns IOC data
3	Flow with IOC-matching IP	Alert severity auto-escalated
4	No IOC match	No escalation, normal flow

TASK 3 — POST /ml/retrain Endpoint 🟢

Time Est: 60 min | **Priority:** 🟢 Medium
Source: MASTER_DOC_PART2 §5.1

Add the retrain endpoint to `backend/app/api/v1/ml.py`. This triggers model retraining as a background task.

3.1 Add Retrain Endpoint

```
import asyncio
import subprocess

class RetrainRequest(BaseModel):
    """Request to trigger model retraining."""
    dataset: str = "nsl_kdd"  # nsl_kdd or cicids2017
    models: list[str] = ["isolation_forest", "random_forest", "autoencoder"]

class RetrainResponse(BaseModel):
    """Response from retrain trigger."""
    status: str
    task_id: str
    message: str

@router.post("/retrain", response_model=RetrainResponse)
async def retrain_models(request: RetrainRequest) -> RetrainResponse:
```

```
    """
    Trigger model retraining (background task).

    Per MASTER_DOC_PART2 §5.1: POST /ml/retrain
    Admin-only endpoint. Launches training in background subprocess.
    """
    import uuid
    task_id = str(uuid.uuid4())[:8]

    # Validate requested models
    valid_models = {"isolation_forest", "random_forest", "autoencoder"}
    invalid = set(request.models) - valid_models
    if invalid:
        raise HTTPException(status_code=400, detail=f"Invalid models: {invalid}")

    # Launch background training
    asyncio.create_task(_run_retraining(task_id, request.dataset, request.models))

    return RetrainResponse(
        status="started",
        task_id=task_id,
        message=f"Retraining {len(request.models)} models on {request.dataset}
dataset",
    )


async def _run_retraining(task_id: str, dataset: str, models: list[str]) -> None:
    """Background retraining task."""
    logger.info("[ML] Retrain started: task=%s, dataset=%s, models=%s", task_id,
dataset, models)
    try:
        proc = await asyncio.create_subprocess_exec(
            "python", "-m", "ml.training.train_all",
            "--dataset", dataset,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )
        stdout, stderr = await proc.communicate()
        if proc.returncode == 0:
            logger.info("[ML] Retrain complete: task=%s", task_id)
        else:
            logger.error("[ML] Retrain failed: task=%s, stderr=%s", task_id,
stderr.decode()[:500])
    except Exception as e:
        logger.error("[ML] Retrain exception: task=%s, error=%s", task_id, e)
```

Verification:

#	Check	Expected
1	POST /ml/retrain with valid body	200, status="started"
2	POST /ml/retrain with invalid model	400 error

#	Check	Expected
3	Background task starts	Log: "[ML] Retrain started"
4	OpenAPI docs show endpoint	/api/v1/ml/retrain visible

TASK 4 — WebSocket: Broadcast new_alert + anomaly_detected 🟢

Time Est: 45 min | **Priority:** 🟢 Medium
Source: MASTER_DOC_PART2 §5.2

The WebSocket infrastructure already exists (`websocket.py` with ConnectionManager + Redis listener). The missing piece is ensuring the ML Worker publishes `anomaly_detected` events to the `ml:live` channel, and that the WebSocket server subscribes to it.

4.1 Add ml:live Channel to WebSocket Server

In `backend/app/api/v1/websocket.py`:

```
# Add to imports (if not present):
CHANNEL_ML_LIVE = "ml:live"

# Add to ConnectionManager.__init__ channel_subscribers:
CHANNEL_ML_LIVE: set(),

# Add to _redis_listener pubsub.subscribe():
CHANNEL_ML_LIVE,
```

4.2 Publish anomaly_detected from ML Worker

In the ML Worker's scoring loop, after an anomaly is detected (score ≥ 0.30):

```
# Publish anomaly_detected event
await redis_client.publish("ml:live", json.dumps({
    "event": "anomaly_detected",
    "payload": {
        "flow_id": flow_id,
        "composite_score": composite_score,
        "severity": severity,
        "category": category,
        "source_ip": src_ip,
        "dest_ip": dst_ip,
        "if_score": if_score,
        "rf_confidence": rf_confidence,
        "ae_score": ae_score,
        "timestamp": datetime.now(timezone.utc).isoformat(),
    }
}))
```

4.3 WebSocket Event Types (per PART2 §5.2)

Channel	Event	Status
`flows:live`	`new_flow`	☑ Already broadcasting
`alerts:live`	`new_alert`	☑ Already broadcasting
`alerts:live`	`alert_updated`	☑ Already has publish helper
`system:status`	`capture_status`	☑ Already broadcasting
`system:status`	`system_metrics`	☑ Already broadcasting
`ml:live`	`anomaly_detected`	● **NEW — Add in Day 13**
`llm:stream`	`token`	⏳ Week 5 (frontend LLM chat)

Verification:

#	Check	Expected
1	WebSocket connects	"connected" event with ml:live in available_channels
2	Subscribe to ml:live	"subscribed" confirmation
3	ML anomaly detected	anomaly_detected event received via WebSocket
4	Alert created	new_alert event received via WebSocket

TASK 5 — Hyperparameter Tuning Execution ◐

Time Est: 45 min | **Priority:** ◐ Medium
Source: MASTER_DOC_PART4 §4.4, §5.3

Create `backend/ml/training/tune_models.py` and execute on VPS.

5.1 Tuning Script

```
"""
ThreatMatrix AI — Hyperparameter Tuning

Grid search over hyperparameters per MASTER_DOC_PART4:
  - Isolation Forest: n_estimators, contamination, max_samples §4.4
  - Random Forest: n_estimators, max_depth, min_samples_split §5.3

NOTE: Results are informational. Current ensemble weights (0.30/0.45/0.25) and
      thresholds (0.90/0.75/0.50/0.30) are LOCKED and MUST NOT change.
"""

# Implementation: GridSearchCV + StratifiedKFold per §7.2
```

```
# Output: best_params.json with optimal hyperparameters
# These can be applied to a future retrain cycle
```

5.2 Execution on VPS

```
docker compose exec ml-worker python -m ml.training.tune_models
```

Verification:

#	Check	Expected
1	Script runs without crash	Completes in <10 min
2	best_params.json generated	Contains optimal params
3	Ensemble weights unchanged	0.30/0.45/0.25 verified
4	Alert thresholds unchanged	0.90/0.75/0.50/0.30 verified

TASK 6 — E2E Test Traffic Generation

Time Est: 30 min | **Priority:** Informational
Source: Validation task

Generate test traffic with nmap/hping3 on VPS to validate the full pipeline.

```
# Port scan (should trigger Probe detection)
nmap -sS -T4 -p 1-1000 187.124.45.161

# SYN flood simulation (should trigger DoS detection)
hping3 -S --flood -p 80 187.124.45.161 --count 1000

# DNS queries
dig @8.8.8.8 example.com
dig @8.8.8.8 google.com
```

Then verify:

```
-- Check new alerts with LLM narratives
SELECT alert_id, severity, category, composite_score,
       LEFT(ai_narrative, 100) as narrative_preview,
       created_at
FROM alerts
WHERE alert_id LIKE 'TM-202%'
ORDER BY created_at DESC
LIMIT 10;
```

Verification:

#	Check	Expected
1	nmap scan captured	New flows in network_flows
2	ML scored the flows	anomaly_score populated
3	Alert generated for anomaly	New alert in alerts table
4	LLM narrative generated	ai_narrative NOT NULL
5	WebSocket event received	anomaly_detected broadcast

Post-Task API Coverage Update

Service	Before	After	Coverage
Auth	5/5	5/5	**100%**
Flows	6/6	6/6	**100%**
Alerts	5/5	5/5	**100%**
Capture	4/5	4/5	80%
System	2/3	2/3	67%
WebSocket	1/1	1/1	**100%**
ML	3/5	**4/5**	**80%** ☑ (+retrain)
LLM	5/5	5/5	**100%**
Intel	4/4	4/4	**100%**
Reports	0/3	0/3	Week 6
TOTAL	**35/42**	**36/42**	**85.7%**

Files Modified / Created (Expected)

File	Action	Lines (est.)
`app/services/alert_engine.py`	MODIFY	+60 (LLM narrative)
`app/services/ioc_correlator.py`	CREATE	~130
`app/api/v1/ml.py`	MODIFY	+50 (retrain endpoint)
`app/api/v1/websocket.py`	MODIFY	+10 (ml:live channel)
`ml/training/tune_models.py`	CREATE	~200
`ml/inference/worker.py`	MODIFY	+15 (anomaly_detected publish)

Risk Assessment

Risk	Impact	Mitigation
LLM rate limiting on burst alerts	Narratives delayed	fire-and-forget + retry
IOC table empty (no API keys)	No correlations found	Graceful degradation
Retrain blocks ML Worker	Inference stalled	Separate subprocess
WebSocket ml:live not subscribed	Frontend misses anomalies	Add channel in Day 13

STRICT RULES REMINDER

1. **DO NOT** change ensemble weights (0.30/0.45/0.25) — LOCKED
2. **DO NOT** change alert thresholds (0.90/0.75/0.50/0.30) — LOCKED
3. **DO NOT** suggest Kafka, Kubernetes, Elasticsearch, MongoDB
4. **DO NOT** add features not in the 10 modules
5. **DO NOT** use Tailwind CSS — Vanilla CSS + CSS Variables only
6. LLM via **OpenRouter only** — 3 verified models
7. Prompts follow **PART4 §9.2** templates
8. Master documentation (5 parts) is the **SOLE source of truth**
9. All code: **typed, error-handled, documented, production-quality**
10. Python: **type hints, async/await, SQLAlchemy 2.x**

Day 13 Worklog — Week 3 Day 4
E2E Pipeline LIVE: capture → ML (22,700+ flows) → alerts → LLM (3 models)
Target: LLM auto-narrative + IOC correlation + retrain endpoint + WebSocket ml:live
Ensemble: 80.73% acc | 80.96% F1 | 0.9312 AUC-ROC (LOCKED)